

A. Ciphertext

题意：有一个大小为 k 的字符集，用 A-Za-z 依次标号。这 k 个字符用 prefix code 来编码。现在有一个字符串 s ，你需要把 s 编码后的字符串划分成最多的段，使得每段都不能解码。

$$1 \leq k \leq 52, 1 \leq |s| \leq 10^6$$

题解：可以发现如果编码里同时有 0 和 1，那么答案显然是 -1；如果同时没有 0 和 1，显然答案是编码后字符串长度。接下来不妨假设有 0 存在，那么显然其他编码都不是以 0 开头的。那么如果我们找到了一个最短的不能被解码的后缀，那么前缀里每个 1 都可以和它前面的一串 0 一起切出去。然后这个后缀可以通过 AC 自动机来求。

B. Graph Subpaths

题意：有一个 n 个点 m 条边的有向无环图。给出其中 k 条路径。

现在对于每个从 2 到 n 的 i ，求出有多少从 1 到 i 的路径，使得任意之前给定的路径不是它的子串。

$$3 \leq n, m \leq 10^5, 1 \leq a_i < b_i \leq n, 0 \leq k \leq 50000, \sum l_i \leq 10^5$$

题解：考虑倒着做，求出 i 到1的路径中有哪些是合法的。假设我们把从 i 到1的路径全部插到一个 trie 里了，那么这个题其实就做完了。令 t_i 是从 i 出发的所有合法路径构成的 trie，不包含节点 i 本身。

按照拓扑顺序依次构建每个 t_i ，假设当前处理到了节点 i ，那么对于所有 i 的出边 j ，我们建一条从 t_i 到 t_j 的 trie 边，边上字符为 j 。这样我们就有了一个从 i 出发的 trie，其中一些从 i 出发的路径可能是不合法的。

接下来需要删掉不合法的路径，对于每条从 i 出发的不合法路径，我们在 trie 上走一遍，把这条路径对应的子树删掉即可。同时 trie 的每个节点维护当前子树的大小。

我们用可持久化数据结构来存储这个 trie，这样就可以做到 $O(n^2)$ 的复杂度。因为每个点可能有 $O(n)$ 条出边。用可持久化线段树存储所有出边就可以把 $O(n)$ 优化成 $O(\log n)$ 了。

别解：考虑对这 k 条路径建立 AC 自动机。令 $dp(i, j)$ 表示当前走到了 DAG 上的点 i ，在自动机上点 j 的合法路径数。可以观察到总得状态数是 $O(n + \sum l_i)$ 的，因为对于一个 i ，

如果走到了 j ，则表明存在一条路径某个前缀以 i 结尾。转移的话直接枚举下一个要走的点 k ，同时在自动机上走即可。

C. Tree Equation

题意：定义了两棵有根树的乘法和加法操作，给出三棵有根树 A ， B 和 C 。求出两棵有根树 x 和 Y ，使得 $A \cdot X + B \cdot Y = C$ 。

$$1 \leq |A|, |B|, |C| \leq 10^5$$

题解：考虑 C 里面最深的叶子是属于 $A \cdot X$ 还是 $B \cdot Y$ 的，不妨考虑是 $A \cdot X$ 的，其他情况类似。

考虑从这个叶子往上走 $height(C) - height(A)$ 步走到一个节点 x ，那么如果有解的话以这个 x 为根的子树一定是 X ，于是我们能够求出 $A \cdot X$ 了。把 $A \cdot X$ 从 C 里删掉，用同样的方法可以求出 Y 。判定下是否合法即可。

Nasty Donchik

题意：给出 N 个整数 $A_1, A_2, \dots, A_N$ ，求出有多少三元组 (i, j, k) 满足

- $1 \leq i \leq j < k \leq N$
- 区间 $[i, j]$ 内数构成的集合和区间 $[j + 1, k]$ 内数构成的集合一样

$$1 \leq N \leq 2 \times 10^5, 1 \leq A_i \leq N$$

题解：先考虑枚举 i, j, k 之后如何判定这个三元组合法。

定义 $next(i)$ 表示 A_i 下一次出现的位置，即 $next(i) = \min\{j \mid j > i, A_j = A_i\}$ ；定义 $prev(i)$ 表示 A_i 上一次出现位置，即 $prev(i) = \max\{j \mid j < i, A_j = A_i\}$ 。

那么 (i, j, k) 合法当且仅当 $\max_{i \leq x \leq j} \{next(x)\} \leq k$ 和 $\min_{j+1 \leq x \leq k} \{prev(x)\} \geq i$ 。

接下来考虑枚举 j 和 k ，然后确定 i 的范围。显然 i 的上界就是 $\min_{j+1 \leq x \leq k} \{prev(x)\}$ ，记为 $up(j, k)$ 。

观察发现，对于某个 i ，如果 $\max_{i \leq x \leq j} \{next(x)\} \leq k$ 成立，那么 $\max_{i+1 \leq x \leq j} \{next(x)\} \leq k$ 也是成立的。于是，满足 $\max_{i \leq x \leq j} \{next(x)\} \leq k$ 的那些 i 其实构成了一个区间。我们可以求出其中最小的 i ，记为 $low(j, k)$ 。

那么答案就是 $\sum_{1 \leq j < k \leq N} upp(j, k) - low(j, k)$ 。

最后考虑之枚举 k ，然后来维护 $upp(j, k)$ 和 $low(j, k)$ 。

固定 k 之后， $upp(j, k)$ 关于 j 的图像其实是一个单调不降的折线，这个我们用一个单调队列 + 线段树是很容易维护的。但是 $low(j, k)$ 似乎不太能直接维护。

观察可以发现，假设对于某个 j ，令 $i = low(j, k)$ ，那么对于每个 $j' \in [i, j]$ ，都有 $low(j', k) = i$ 。并且肯定有 $next(i - 1) > k$ 。

于是可以发现，考虑找出 $next(i_1), next(i_2), \dots, next(i_m) > k$ ， $1 \leq i_1 < i_2 < \dots < i_m$ ，对于 $j \in [i_x, i_{x+1}]$ ，都有 $low(j, k) = i_x$ 。于是 $low(j, k)$ 的图像其实也是一个单调不降的折线。

然后每次从 k 变到 $k + 1$ 的时候，最多只有一个 i_x 会改变，就是 $i_x = prev(k + 1)$ 的那个 i_x 。用一个线段树也是可以轻松维护的。

对于每个 k ，我们要求 $sum = \sum_{1 \leq j < k} upp(j, k) - low(j, k)$ 。这两条折线我们已经用线段树维护了，因此只要在修改的时候去对方线段树上查个区间和就能 $O(\log n)$ 维护这个 sum 。